

Máquina Norma: resoluções completas dos exemplos e exercícios

Material com a resolução explicada dos exemplos convertidos em exercícios e dos exercícios do capítulo sobre Máquinas de Registradores - Norma. As soluções usam a codificação por números primos, a representação de instruções por quádruplas e as macros de Máquina Norma.

Observação: nos algoritmos em Norma, as macros usadas podem ser expandidas por definições anteriores. Por exemplo, $A:=0$, $A:=B$ usando C , $A:=A+B$ usando C , teste $A<B$ e teste_mod(A,B) são macros construídas a partir das operações básicas adK, subK e zeroK.

Regras usadas nas resoluções

A Máquina Norma trabalha com registradores naturais e três recursos primitivos: incrementar um registrador, decrementar um registrador sem ficar negativo e testar se um registrador é zero. A partir desses recursos, definimos macros para operações mais complexas.

Na codificação de n-uplas, usamos o Teorema Fundamental da Aritmética: cada número natural possui uma decomposição única em fatores primos. Assim, uma n-upla $(x_1, x_2, \dots, x_n)$ é codificada por $2^{x_1} \cdot 3^{x_2} \cdot 5^{x_3} \cdot 7^{x_4} \dots$.

Na codificação de programas monolíticos, cada instrução rotulada é primeiro transformada em uma quádrupla:

- Operação: r1: faça Fk vá_para r2 vira (0, k, r2, r2).
- Teste: r1: se Tk então vá_para r2 senão vá_para r3 vira (1, k, r2, r3).

Cada quádrupla (a,b,c,d) é codificada como $2^a \cdot 3^b \cdot 5^c \cdot 7^d$. Depois, se o programa possui m instruções, a codificação final é $2^{i_1} \cdot 3^{i_2} \cdot 5^{i_3} \cdot 7^{i_4} \dots$, em que $i_1, i_2, \dots$ são os códigos das instruções.

Parte A - Exemplos do PDF resolvidos

Exemplo 4.1 - Codificação de n-uplas por número de Gödel

Ideia da solução: representar uma lista finita de números naturais como um único número natural. Cada posição da lista é armazenada como expoente de um primo diferente. A primeira posição usa o primo 2, a segunda usa 3, a terceira usa 5, e assim por diante.

Escolhendo a 4-upla (2, 0, 3, 1), temos:

$$\begin{aligned}c(2,0,3,1) &= 2^2 \cdot 3^0 \cdot 5^3 \cdot 7^1 \\&= 4 \cdot 1 \cdot 125 \cdot 7 \\&= 3500\end{aligned}$$

Para decodificar 3500, fatoramos:

$$3500 = 2^2 \cdot 3^0 \cdot 5^3 \cdot 7^1$$

Logo, recuperamos a 4-upla (2, 0, 3, 1). A codificação é injetora porque duas n-uplas diferentes produzem decomposições diferentes em fatores primos. Ela não é necessariamente bijetora porque nem todo número natural usa apenas os n primeiros primos da forma esperada. Por exemplo, no caso de 3-uplas, um número com fator 7 não estaria no formato $2^x \cdot 3^y \cdot 5^z$.

Exemplo 4.2 - Codificação de programas monolíticos

Ideia da solução: transformar cada instrução em uma quádrupla e depois transformar cada quádrupla em um número. O programa inteiro vira uma upla de códigos de instruções.

Considere o programa simples:

```
1: faça F1 vá_para 2
2: se T1 então vá_para 1 senão vá_para 0
```

A instrução 1 é uma operação F1 que vai para 2, então vira a quádrupla (0,1,2,2).

$$i1 = 2^0 \cdot 3^1 \cdot 5^2 \cdot 7^2 = 3675$$

A instrução 2 é um teste T1, verdadeiro para 1 e falso para 0, então vira (1,1,1,0).

$$i2 = 2^1 \cdot 3^1 \cdot 5^1 \cdot 7^0 = 30$$

Logo, a codificação do programa é:

$$P = 2^{3675} \cdot 3^{30}$$

Exemplo 4.3 - Codificação do programa monolítico Q

Ideia da solução: como o programa possui oito instruções, calculamos oito códigos de instrução i1 até i8. Depois usamos os oito primeiros primos como base da codificação do programa.

Rótulo	Instrução	Quádrupla	Código da instrução
1	faça F vá para 2	(0, 1, 2, 2)	$2^0 \cdot 3^1 \cdot 5^2 \cdot 7^2 = 3675$
2	se T então vá para 3 senão vá para 5	(1, 1, 3, 5)	$2^1 \cdot 3^1 \cdot 5^3 \cdot 7^5 = 12605250$
3	faça G vá para 4	(0, 2, 4, 4)	$2^0 \cdot 3^2 \cdot 5^4 \cdot 7^4 = 13505625$
4	se T então vá para 1 senão vá para 0	(1, 1, 1, 0)	$2^1 \cdot 3^1 \cdot 5^1 \cdot 7^0 = 30$
5	faça F vá para 6	(0, 1, 6, 6)	$2^0 \cdot 3^1 \cdot 5^6 \cdot 7^6 = 5514796875$
6	se T então vá para 7 senão vá para 2	(1, 1, 7, 2)	$2^1 \cdot 3^1 \cdot 5^7 \cdot 7^2 = 22968750$
7	faça G vá para 8	(0, 2, 8, 8)	$2^0 \cdot 3^2 \cdot 5^8 \cdot 7^8 = 20266878515625$
8	se T então vá para 6 senão vá para 0	(1, 1, 6, 0)	$2^1 \cdot 3^1 \cdot 5^6 \cdot 7^0 = 93750$

Portanto:

$$q = 2^{3675} \cdot 3^{51450} \cdot 5^{13505625} \cdot 7^{30} \cdot 11^{49633171875} \cdot 13^{102941874} \cdot 17^{4135546875} \cdot 19^{93750}$$

Observação: alguns materiais mantêm os códigos das instruções na forma fatorada, por exemplo $i1 = 2^0 \cdot 3^1 \cdot 5^2 \cdot 7^2$, em vez de calcular o valor decimal.

Exemplo 4.4 - Decodificação de programa monolítico

Dado $p = 2^{150} \cdot 3^{105}$, sabemos que o programa possui duas instruções, pois aparecem os primos 2 e 3 como bases externas. Os expoentes 150 e 105 são os códigos das instruções.

$$150 = 2^1 \cdot 3^1 \cdot 5^2 \cdot 7^0 \rightarrow \text{quádrupla } (1,1,2,0)$$

$$105 = 2^0 \cdot 3^1 \cdot 5^1 \cdot 7^1 \rightarrow \text{quádrupla } (0,1,1,1)$$

A primeira quádrupla é teste, pois começa com 1. A segunda é operação, pois começa com 0.

1: se T1 então vá para 2 senão vá para 0

2: faça F1 vá para 1

Exemplo 4.5 - Atribuição do valor zero ao registrador A

Lógica da solução: A única forma de reduzir um registrador natural em Norma é aplicar subA repetidamente. Como subA nunca deixa o valor ficar negativo, basta repetir $A := A - 1$ até que o teste $A = 0$ seja verdadeiro. Quando A chega a zero, o processo para.

Algoritmo iterativo:

ate $A = 0$ faça

$A := A - 1$

Instruções rotuladas:

1: se zeroA então vá para 3 senão vá para 2

2: faça subA vá para 1

Exemplo 4.6 - Atribuição de um valor natural a um registrador

Lógica da solução: Para atribuir um valor exato n a A, primeiro zeramos A. Depois aplicamos n incrementos. Se não zerarmos A antes, o valor final será o valor antigo de A somado a n, e não exatamente n.

Macro geral $A := n$:

$A := 0$

repita n vezes:

```
A := A + 1
```

```
Caso A := 3:  
A := 0  
A := A + 1  
A := A + 1  
A := A + 1
```

Exemplo 4.7 - Adição de dois registradores sem preservar B

Lógica da solução: Para somar B em A, transferimos uma unidade de B para A a cada iteração. Cada passo incrementa A e decrementa B. Quando B chega a zero, todas as unidades originais de B foram incorporadas em A. O preço é que B é destruído, pois termina com zero.

```
Algoritmo A := A + B:  
ate B = 0 faca  
    A := A + 1  
    B := B - 1
```

Resultado: A recebe $A_{original} + B_{original}$; B termina com 0.

Exemplo 4.8 - Adição preservando B

Lógica da solução: A estratégia anterior destrói B. Para preservar B, usamos um auxiliar C. Enquanto transferimos unidades de B para A, também copiamos essas unidades para C. Depois restauramos B transferindo as unidades de C de volta para B.

```
Algoritmo A := A + B usando C:  
C := 0  
ate B = 0 faca  
    A := A + 1  
    C := C + 1  
    B := B - 1  
ate C = 0 faca  
    B := B + 1  
    C := C - 1
```

Resultado: A recebe a soma, B é restaurado e C termina zerado.

Exemplo 4.9 - Atribuição do conteúdo de um registrador

Lógica da solução: Para fazer $A := B$ sem destruir B, primeiro zeramos A. Depois usamos a soma preservando B. Como A começa em zero, $A := A + B$ equivale a $A := B$.

```
Algoritmo A := B usando C:  
A := 0  
A := A + B usando C
```

Expansão conceitual:

```
A := 0  
C := 0  
ate B = 0 faca  
    A := A + 1  
    C := C + 1  
    B := B - 1  
ate C = 0 faca  
    B := B + 1  
    C := C - 1
```

Exemplo 4.10 - Multiplicação de dois registradores

Lógica da solução: Multiplicar A por B significa somar B ao resultado $A_{original}$ vezes. Como A contém inicialmente o número de repetições, primeiro copiamos o valor inicial de A para C, zerando A. Depois, enquanto C não for zero, somamos B em A e decrementamos C. O registrador D é usado para preservar B durante cada soma.

```

Algoritmo A := A . B usando C,D:
C := 0
ate A = 0 faca
    C := C + 1
    A := A - 1
ate C = 0 faca
    A := A + B usando D
    C := C - 1

```

Expansão da soma preservando B:

```

C := 0
ate A = 0 faca
    C := C + 1
    A := A - 1
ate C = 0 faca
    D := 0
    ate B = 0 faca
        A := A + 1
        D := D + 1
        B := B - 1
    ate D = 0 faca
        B := B + 1
        D := D - 1
    C := C - 1

```

Exemplo 4.11 - Teste se $A < B$

Lógica da solução: Para comparar A e B, copiamos A para C e B para D. Depois removemos uma unidade de C e uma unidade de D simultaneamente. Se D chegar a zero antes de C, então B não era maior que A. Se C chegar a zero enquanto D ainda possui valor positivo, então $A < B$.

```

Teste A < B usando C,D,E:
C := A usando E
D := B usando E
ate C = 0 faca
    se D = 0 então
        retorne falso
    senão
        C := C - 1
        D := D - 1
se D = 0 então
    retorne falso
senão
    retorne verdadeiro

```

```

A B C D E    2 < 3 LOGO A < B É True
2 3 2 3 0
1 2
0 1

```

Logo retorna verdadeiro. Basta setar por exemplo o registrador E com valor 1 (add do E que já vale zero) ou do contrario por zero no E (ele já tem o valor zero, logo não precisa fazer nada)

Exemplo 4.12 - Teste se o resto da divisão inteira é zero

Lógica da solução: Queremos saber se A é múltiplo de B. **Se $B=0$, o teste deve retornar falso.** Se $A=0$ e $B>0$, o resto é zero, então retorna verdadeiro. Nos demais casos, copiamos A para C e vamos subtraindo B de C enquanto $C \geq B$. Ao final, se $C=0$, o resto foi zero; caso contrário, sobrou um resto não nulo.

Para saber se **A é múltiplo de B**, queremos verificar se a divisão inteira de A por B deixa **resto zero**.

Como a Máquina Norma trabalha com operações simples, como **somar 1**, **subtrair 1** e **testar zero**, a divisão é simulada por **subtrações repetidas**.

Ou seja:

$A \bmod B =$ resto da divisão de A por B

Em vez de calcular diretamente:

A / B

fazemos:

$A - B - B - B - \dots$

até não ser mais possível subtrair B.

Se no final sobrar 0, então A é múltiplo de B.

Se no final sobrar algum valor diferente de 0, então A **não é múltiplo** de B.

Por que subtrair B de C?

O algoritmo faz:

$C := A$

porque queremos preservar o valor original de A.

Então usamos C como uma cópia de trabalho.

Depois fazemos:

$C := C - B$

$C := C - B$

$C := C - B$

$\dots$

Cada subtração representa tirar **um grupo completo de tamanho B** de dentro de A.

Por exemplo, se queremos testar:

4 é múltiplo de 2?

pensamos assim:

$4 - 2 = 2$

$2 - 2 = 0$

Como chegou exatamente em 0, então 4 é múltiplo de 2.

Agora, se queremos testar:

5 é múltiplo de 3?

fazemos:

$$5 - 3 = 2$$

Agora não dá mais para subtrair 3, porque $2 < 3$.

Sobrou 2.

Logo, o resto é 2, então 5 **não é múltiplo** de 3.

Tabela da execução: A = 4, B = 2

Passo	Operação realizada	A	B	C	E	Comentário
1	Início	4	2	0	0	Valores iniciais
2	C := A	4	2	4	0	Copiamos A para C
3	Testa B = 0	4	2	4	0	Não, pois B vale 2
4	Testa A = 0	4	2	4	0	Não, pois A vale 4
5	Testa B <= C	4	2	4	0	Sim, pois $2 \leq 4$
6	C := C - B	4	2	2	0	C passa de 4 para 2
7	Testa B <= C	4	2	2	0	Sim, pois $2 \leq 2$
8	C := C - B	4	2	0	0	C passa de 2 para 0
9	Testa B <= C	4	2	0	0	Não, pois $2 \leq 0$ é falso
10	Testa C = 0	4	2	0	0	Sim
11	Retorna verdadeiro	4	2	0	0	4 é múltiplo de 2

Tabela da execução: A = 5, B = 3

Passo	Operação realizada	A	B	C	E	Comentário
1	Início	5	3	0	0	Valores iniciais
2	C := A	5	3	5	0	Copiamos A para C
3	Testa B = 0	5	3	5	0	Não, pois B vale 3
4	Testa A = 0	5	3	5	0	Não, pois A vale 5
5	Testa B <= C	5	3	5	0	Sim, pois 3 <= 5
6	C := C - B	5	3	2	0	C passa de 5 para 2
7	Testa B <= C	5	3	2	0	Não, pois 3 <= 2 é falso
8	Testa C = 0	5	3	2	0	Não
9	Retorna falso	5	3	2	0	5 não é múltiplo de 3

```

Teste teste_mod(A,B):
C := A usando E
se B = 0 então
    retorne falso
senão se A = 0 então
    retorne verdadeiro
senão
    enquanto B <= C faça
        C := C - B usando E
    se C = 0 então
        retorne verdadeiro
    senão
        retorne falso

```

4 / 2 da resto zero Logo eh multiplo
5 / 3 da resto 2

4 / 2 da resto zero

A B C D

4 2 4

Exemplo 4.13 - Teste se A é primo

Lógica da solução: Um número é primo quando possui exatamente dois divisores positivos: 1 e ele mesmo. No material, considera-se que 1 é tratado como primo para fins da enumeração posterior. A ideia é testar divisores menores que A. Copiamos A para C, decrementamos C e verificamos se algum divisor C divide A. Se encontrarmos divisor próprio, A não é primo; caso contrário, é primo.

No entanto, o material diz que **1 será tratado como primo para fins da enumeração posterior**.

Isso é uma **convenção didática ou técnica do material**, não a definição matemática padrão.

suponha que 1 é o 0-ésimo número primo, apesar de não ser primo.

O algoritmo quer testar se A é primo.

A ideia é:

Verificar se existe algum divisor de A entre $A - 1$ e 2.

Porque todo número A sempre é divisível por:

1
 A

Esses divisores são chamados de divisores triviais.

Para saber se A é primo, precisamos descobrir se existe algum divisor próprio, isto é, algum divisor diferente de 1 e diferente de A .

Exemplo:

$A = 7$ (7 é primo?)

Testamos possíveis divisores menores que 7:

6, $(A-1)$, 5, 4, 3, 2, 1

Nenhum deles divide 7, exceto 1.

Então 7 é primo.

Agora:

$A = 6$ (6 é divisível por 3, por 2, logo não é primo)

Testamos:

5, 4, 3, 2, 1

O número 3 divide 6.

Então 6 não é primo.

O papel do registrador C

O registrador `c` é usado como candidato a divisor.

O algoritmo faz:

```
C := A  
C := C - 1
```

Então `c` passa a valer:

```
A - 1
```

Por exemplo, se:

```
A = 7
```

então:

```
C := 7  
C := C - 1  
C := 6
```

Agora o algoritmo vai testar se 6 divide 7.

Se não dividir, diminui `c`.

Depois testa 5, depois 4, depois 3, depois 2, depois 1.

O que significa `teste_mod(A,C)`

A macro:

```
teste_mod(A,C)
```

retorna `verdadeiro` se o resto da divisão inteira de `A` por `C` for zero.

Ou seja:

```
teste_mod(A,C) = verdadeiro
```

significa:

```
C divide A
```

xemplos:

```
teste_mod(6,3) = verdadeiro
```

porque:

```
6 / 3 = 2, resto 0
```

Mas:

```
teste_mod(7,3) = falso
```

porque:

```
7 / 3 = 2, resto 1
```

```
ate teste_mod(A,C) faça  
  C := C - 1  
Enquanto C não for divisor de A, diminua C.  
Pare quando encontrar um divisor de A.
```

Por que depois do até aparece novamente `C := C - 1`?

Depois que o comando até termina, significa que o algoritmo encontrou um divisor de A.

Ou seja, quando sai do laço, temos:

```
teste_mod(A,C) = verdadeiro
```

Portanto, o valor atual de C divide A.

Agora vem a pergunta principal:

O divisor encontrado **foi 1 ou foi um divisor maior que 1?**

Se o divisor encontrado **foi 1, então o número é primo.**

Se o divisor encontrado **foi maior que 1, então o número não é primo.**

O algoritmo faz isto:

```
C := C - 1  
se C = 0 então verdadeiro  
senão falso
```

Isso é uma forma indireta de testar se C era igual a 1.

Porque:

```
se C era 1
então C := C - 1
C vira 0
```

Logo:

$C = 0$

significa:

o divisor encontrado era 1

Portanto, A é primo.

Mas se o divisor encontrado era, por exemplo, 3:

```
C := 3
C := C - 1
C := 2
```

Agora C não é zero.

Logo, o divisor encontrado era maior que 1.

Portanto, A não é primo.

Uma versão mais didática seria:

teste_primo(A)

se $A = 0$ então
retorne falso

$C := A - 1$

se $C = 0$ então
retorne verdadeiro
// isso faz $A = 1$ ser considerado primo no material

enquanto C não divide A faça
 $C := C - 1$

// quando sai do laço, C é um divisor de A

se $C = 1$ então
retorne verdadeiro
senão
retorne falso

Como a Máquina Norma não testa diretamente $C = 1$, ela faz:

```
C := C - 1  
  
se C = 0 então  
    retorne verdadeiro  
senão  
    retorne falso
```

```
Teste teste_primo(A):  
se A = 0 então  
    retorne falso  
C := A  
C := C - 1  
se C = 0 então  
    retorne verdadeiro  
ate teste_mod(A,C) faça  
    C := C - 1  
C := C - 1  
se C = 0 então  
    retorne verdadeiro  
senão  
    retorne falso
```

Leitura didática: percorra candidatos a divisor e verifique se algum divide A sem resto.

Exemplo passo a passo: A = 7

Vamos testar se 7 é primo.

Valores iniciais:

```
A = 7  
C = 0
```

Passo	A	C	Teste feito	Resultado	Comentário
1	7	6	<code>teste_mod(7,6)</code>	falso	6 não divide 7
2	7	5	<code>teste_mod(7,5)</code>	falso	5 não divide 7
3	7	4	<code>teste_mod(7,4)</code>	falso	4 não divide 7
4	7	3	<code>teste_mod(7,3)</code>	falso	3 não divide 7
5	7	2	<code>teste_mod(7,2)</code>	falso	2 não divide 7
6	7	1	<code>teste_mod(7,1)</code>	verdadeiro	1 divide 7

Exemplo passo a passo: A = 6

Agora vamos testar se 6 é primo.

Valores iniciais:

A = 6
C = 0

Passo	A	C	Teste feito	Resultado	Comentário
1	6	5	<code>teste_mod(6,5)</code>	falso	5 não divide 6
2	6	4	<code>teste_mod(6,4)</code>	falso	4 não divide 6
3	6	3	<code>teste_mod(6,3)</code>	verdadeiro	3 divide 6

Como ficaria se quiséssemos a definição matemática correta?

Se quisermos seguir a definição matemática moderna, deveríamos fazer:

```
se A = 0 então
  retorne falso
senão se A = 1 então
  retorne falso
```

```
senão
  teste os divisores
```

Como a Máquina Norma testa zero, para testar $A = 1$, poderíamos fazer uma cópia de A em C , subtrair 1 e testar se virou zero.

A ideia seria:

```
C := A
C := C - 1

se C = 0 então
  retorne falso
```

Isso faria:

```
A = 1
```

retornar falso.

Depois, para os demais números, o teste continuaria normalmente.

14. Resumo final

O algoritmo funciona assim:

1. Se $A = 0$, retorna falso.
2. Copia A para C .
3. Faz $C = A - 1$.
4. Se $C = 0$, então A era 1.
O material retorna verdadeiro por convenção.
5. Caso contrário, procura um divisor de A começando em $A - 1$ e descendo até 1.
6. Quando encontra um divisor C :
se $C = 1$, então A é primo;
se $C > 1$, então A não é primo.

O trecho aparentemente estranho:

```
C := C - 1
se C = 0
```

serve para testar indiretamente se:

```
C = 1
```

Porque a Máquina Norma não testa diretamente se um registrador vale 1; ela testa se ele vale 0.

Então:

```
C = 1
```

é verificado fazendo:

$C := C - 1$

e depois:

$C = 0?$

Se sim, então o divisor encontrado era 1, e o número é considerado primo pelo algoritmo.

Exemplo 4.14 - Atribuição do n-ésimo número primo

Lógica da solução: O algoritmo procura primos sucessivos. A começa em 1, considerado o 0-ésimo primo no material. D recebe B, que indica quantos avanços devem ser feitos. A cada avanço, incrementamos A até encontrar o próximo primo.

Algoritmo A := primo(B) usando D:

A := 1

D := B

ate D = 0 faca

 D := D - 1

 A := A + 1

 ate teste_primo(A) faca

 A := A + 1

Resultado: A contém o B-ésimo primo segundo a convenção adotada.

A ideia é:

Começar em $A = 1$ e avançar B vezes para o próximo número considerado primo.

Como o material considera 1 como o **0-ésimo número primo**, a sequência usada pelo algoritmo fica assim:

Índice	Valor considerado primo
0º	1
1º	2
2º	3
3º	5
4º	7
5º	11
6º	13

Então:

primo(0) = 1

primo(1) = 2

primo(2) = 3

```
primo(3) = 5  
primo(4) = 7
```

Matematicamente, o primeiro primo é realmente 2. O material não está dizendo que 1 é primo no sentido matemático moderno. Ele está criando uma convenção operacional:

1 será tratado como o valor inicial da enumeração, chamado de 0-ésimo primo.

Isso permite que o programa comece em $A = 1$ e, a cada repetição, avance para o próximo primo.

Então, quando o programa recebe:

$B = 1$ (o primeiro primo é o 2)

ele deve retornar:

$A = 2$

Quando recebe:

$B = 2$

ele deve retornar:

$A = 3$

Quando recebe:

$B = 3$

ele deve retornar:

$A = 5$

Ou seja, no material:

$A := \text{primo}(B)$

significa:

A recebe o B-ésimo valor da sequência 1, 2, 3, 5, 7, 11, ...

Por que começar com $A := 1$?

O programa começa assim:

```
A := 1;  
D := B;
```

O registrador A guarda o número primo encontrado até o momento.

Inicialmente:

$A = 1$

Isso significa:

Antes de avançar qualquer vez, estou no 0-ésimo primo.

O registrador D guarda quantas vezes ainda precisamos avançar para o próximo primo.

Se:

$B = 0$

então:

$D = 0$

O laço não executa nenhuma vez e o resultado fica:

$A = 1$

Portanto:

$\text{primo}(0) = 1$

Se:

$B = 1$

então o programa avança uma vez a partir de 1 e chega em 2.

Se:

$B = 2$

então o programa avança duas vezes:

$1 \rightarrow 2 \rightarrow 3$

Se:

$B = 3$

então o programa avança três vezes:

$1 \rightarrow 2 \rightarrow 3 \rightarrow 5$

3. Para que serve $D := B$?

O registrador D é o contador de repetições.

O programa quer avançar exatamente B vezes.

Como a Máquina Norma geralmente trabalha decrementando até chegar a zero, o algoritmo faz:

```
D := B
```

Depois, dentro do laço:

```
D := D - 1
```

Então D vai diminuindo:

$B, B-1, B-2, \dots, 0$

Quando $D = 0$, o programa para.

Portanto:

D controla quantas vezes o algoritmo deve procurar o próximo primo.

4. O que faz o laço externo?

O trecho:

```
até D = 0
  faça (
    D := D - 1;
    A := A + 1;

    até teste_primo(A)
    faça (
      A := A + 1
    )
  )
```

pode ser lido assim:

```
Enquanto D ainda não chegou a zero:
  diminua D em 1;
  avance A para o próximo número;
  enquanto A não for primo:
    avance A mais uma vez.
```

Em pseudocódigo mais comum:

```
A := 1
D := B
```

```
enquanto D ≠ 0 faça
    D := D - 1

    A := A + 1

    enquanto A não é primo faça
        A := A + 1

retorne A
```

A cada repetição do laço externo, o algoritmo sai do primo atual e procura o próximo primo.

5. O que faz o laço interno?

O laço interno é:

```
até teste_primo(A)
faça (
    A := A + 1
)
```

Ele significa:

Enquanto A ainda não for primo, aumente A em 1.

Ou seja, ele procura o próximo número que passa no teste de primalidade.

Exemplo: suponha que $A = 4$.

O algoritmo testa:

```
teste_primo(4)
```

Resultado:

```
falso
```

Então faz:

```
A := A + 1
```

Agora:

```
A = 5
```

Testa novamente:

```
teste_primo(5)
```

Resultado:

verdadeiro

Então o laço interno para.

Conclusão:

A saiu de 4 e chegou ao próximo primo, que é 5.

6. Exemplo 1: calcular primo(0)

Queremos:

```
A := primo(0)
```

Entrada:

```
B = 0
```

Inicialização:

```
A := 1  
D := B
```

Então:

```
A = 1  
D = 0
```

Agora o algoritmo testa o laço externo:

```
D = 0?
```

Sim.

Então o laço não executa.

Resultado final:

```
A = 1
```

Portanto:

```
primo(0) = 1
```

Esse é exatamente o motivo de o material dizer que 1 é o 0-ésimo número primo.

7. Exemplo 2: calcular primo(1)

Agora queremos:

```
A := primo(1)
```

Entrada:

```
B = 1
```

Inicialização:

```
A := 1  
D := B
```

Então:

```
A = 1  
D = 1
```

Agora começa o laço externo.

Primeira repetição

Testa:

```
D = 0?
```

Não, porque:

```
D = 1
```

Então executa:

```
D := D - 1
```

Logo:

```
D = 0
```

Depois:

```
A := A + 1
```

Logo:

```
A = 2
```

Agora entra no laço interno:

```
até teste_primo(A)
```

Testa:

```
teste_primo(2)
```

Resultado:

```
verdadeiro
```

Como 2 é primo, o laço interno não precisa incrementar mais nada.

Estado final:

```
A = 2  
D = 0
```

O laço externo testa novamente:

```
D = 0?
```

Sim.

Então para.

Resultado:

```
primo(1) = 2
```

Portanto, para o material:

```
1º primo = 2
```

8. Exemplo 3: calcular primo(2)

Agora queremos:

```
A := primo(2)
```

Entrada:

```
B = 2
```

Inicialização:

```
A = 1  
D = 2
```

O algoritmo precisa avançar duas vezes:

```
1 → 2 → 3
```

Primeira repetição do laço externo

Estado antes:

A = 1
D = 2

Testa:

D = 0?

Não.

Executa:

D := D - 1

Então:

D = 1

Executa:

A := A + 1

Então:

A = 2

Testa:

teste_primo(2)

Resultado:

verdadeiro

Então para o laço interno.

Estado ao final da primeira repetição:

A = 2
D = 1

Interpretação:

Já avançamos uma vez.
Saímos do 0-ésimo primo, que era 1, e chegamos ao 1º primo, que é 2.

Segunda repetição do laço externo

Estado antes:

A = 2
D = 1

Testa:

D = 0?

Não.

Executa:

D := D - 1

Então:

D = 0

Executa:

A := A + 1

Então:

A = 3

Testa:

teste_primo(3)

Resultado:

verdadeiro

Então o laço interno para.

Estado final:

A = 3
D = 0

Agora o laço externo termina.

Resultado:

primo(2) = 3

9. Exemplo 4: calcular primo(3)

Agora queremos:

A := primo(3)

Entrada:

B = 3

Inicialização:

A = 1

D = 3

O algoritmo precisa avançar três vezes:

1 → 2 → 3 → 5

Tabela passo a passo:

Repetição	D antes	A antes	Faz D := D - 1	Faz A := A + 1	Teste de primo	A final da repetição
1	3	1	D = 2	A = 2	teste_primo(2) = verdadeiro	2
2	2	2	D = 1	A = 3	teste_primo(3) = verdadeiro	3
3	1	3	D = 0	A = 4	teste_primo(4) = falso; incrementa para 5	5

Vamos detalhar a terceira repetição, porque ela mostra bem o funcionamento.

No início da terceira repetição:

A = 3

D = 1

Executa:

D := D - 1

Então:

D = 0

Executa:

A := A + 1

Então:

A = 4

Agora testa:

```
teste_primo(4)
```

Resultado:

```
falso
```

Porque 4 não é primo.

Então o laço interno faz:

```
A := A + 1
```

Logo:

```
A = 5
```

Testa:

```
teste_primo(5)
```

Resultado:

```
verdadeiro
```

Então para.

Resultado final:

```
A = 5
```

Portanto:

```
primo(3) = 5
```

10. Exemplo 5: calcular primo(4)

Agora queremos:

```
A := primo(4)
```

Entrada:

```
B = 4
```

Inicialização:

```
A = 1  
D = 4
```

O algoritmo precisa avançar quatro vezes:

1 → 2 → 3 → 5 → 7

Tabela:

Repetição D antes A antes Primeiro candidato O candidato é primo? Ajustes feitos A final

1	4	1	2	Sim	Nenhum	2
2	3	2	3	Sim	Nenhum	3
3	2	3	4	Não	Avança para 5	5
4	1	5	6	Não	Avança para 7	7

Resultado:

`primo(4) = 7`

11. Exemplo mais completo: calcular primo(5)

Agora queremos:

`A := primo(5)`

Entrada:

`B = 5`

Inicialização:

`A = 1`
`D = 5`

O algoritmo precisa avançar cinco vezes:

1 → 2 → 3 → 5 → 7 → 11

Tabela:

Repetição D antes A antes Candidato inicial após `A := A + 1` Busca pelo próximo primo A final

1	5	1	2	2 é primo	2
2	4	2	3	3 é primo	3
3	3	3	4	4 não é primo; 5 é primo	5
4	2	5	6	6 não é primo; 7 é primo	7

Repetição D antes A antes Candidato inicial após $A := A + 1$ Busca pelo próximo primo A final

5 1 7 8 8 não; 9 não; 10 não; 11 sim 11

Resultado:

`primo(5) = 11`

12. Por que ele chega na resposta certa?

Porque o algoritmo mantém a seguinte ideia:

A guarda o primo atual.
D guarda quantos avanços ainda faltam.

Inicialmente:

$A = 1$

Ou seja:

A está no 0-ésimo valor da sequência.

A cada repetição:

$D := D - 1$

indica que uma etapa da busca será feita.

Depois:

$A := A + 1$

faz o programa sair do valor atual e testar o próximo número.

Depois:

até teste_primo(A)
faça $A := A + 1$

faz o programa ignorar os números que não são primos.

Assim, cada repetição do laço externo move A para o próximo primo da sequência.

Exemplo:

$A = 1$

Primeira repetição:

A vai para 2

Segunda repetição:

A vai para 3

Terceira repetição:

A tenta 4, mas 4 não é primo, então vai para 5

Quarta repetição:

A tenta 6, mas 6 não é primo, então vai para 7

Quinta repetição:

A tenta 8, depois 9, depois 10, até chegar em 11

Portanto, depois de B repetições, A estará no B -ésimo valor da sequência.

13. Por que usar 1 como 0-ésimo primo?

Existem dois motivos principais.

Primeiro motivo: permitir entrada $B = 0$

Se $B = 0$, o algoritmo retorna imediatamente:

$A = 1$

Isso define uma base inicial para a enumeração.

A sequência fica:

```
primo(0) = 1
primo(1) = 2
primo(2) = 3
primo(3) = 5
primo(4) = 7
```

Essa ideia é parecida com sequências que começam no índice zero, como acontece em programação.

Por exemplo, em muitas linguagens:

`vetor[0]` é o primeiro elemento físico do vetor

Aqui o material faz algo semelhante:

`primo(0)` é o valor inicial da enumeração

Mas esse valor inicial é artificial.

Segundo motivo: simplificar o algoritmo

Começar em $A = 1$ facilita a lógica:

```
A := 1  
D := B
```

Depois basta repetir B vezes:

avançar para o próximo primo

Se o algoritmo começasse diretamente em $A = 2$, teria que tratar separadamente o caso do primeiro primo.

Por exemplo:

```
se B = 1 então A := 2  
senão procurar os próximos
```

Ao usar 1 como 0-ésimo valor, o algoritmo evita esse caso especial.

14. Mas isso está matematicamente correto?

Com a definição usual de número primo, não.

Matematicamente:

1 não é primo.

O correto seria:

```
1° primo = 2  
2° primo = 3  
3° primo = 5  
4° primo = 7  
5° primo = 11
```

O material usa uma convenção computacional:

```
0° valor = 1  
1° primo real = 2  
2° primo real = 3  
3° primo real = 5
```

Então, em aula, eu recomendaria explicar assim:

O algoritmo considera 1 como 0-ésimo primo apenas para facilitar a enumeração. Isso não altera o fato matemático de que 1 não é primo. A partir do índice 1, os valores retornados coincidem com os primos usuais.

15. Como ficaria se quiséssemos usar a definição usual?

Se quisermos que:

1° primo = 2
2° primo = 3
3° primo = 5
4° primo = 7

então podemos interpretar a função do material assim:

`primo_material(B) = primo_matemático(B), para  $B \geq 1$`

Ou seja:

B Resultado do material Interpretação matemática

0 1	Valor artificial inicial
1 2	1° primo real
2 3	2° primo real
3 5	3° primo real
4 7	4° primo real
5 11	5° primo real

Portanto, se o usuário passar $B = 1$, o algoritmo retorna o primeiro primo real.

Se passar $B = 2$, retorna o segundo primo real.

Se passar $B = 3$, retorna o terceiro primo real.

Então, apesar de incluir 1 como caso artificial de índice zero, o algoritmo ainda funciona corretamente para $B \geq 1$.

16. Explicação curta do algoritmo

O algoritmo faz isto:

```
A começa em 1.  
D recebe B.  
Enquanto D não for zero:  
    diminui D em 1;  
    aumenta A em 1;  
    enquanto A não for primo:  
        aumenta A em 1.  
Ao final, A contém o B-ésimo valor da sequência 1, 2, 3, 5, 7, 11, ...
```

Exemplo para $B = 4$:

```
A começa em 1  
  
1ª repetição: A vai para 2  
2ª repetição: A vai para 3  
3ª repetição: A vai para 5  
4ª repetição: A vai para 7
```

Resultado: $A = 7$

Logo:

$\text{primo}(4) = 7$

Na interpretação matemática usual:

7 é o 4º primo real.

17. Resumo final

O ponto principal é:

O algoritmo não começa no primeiro primo real.
Ele começa em um valor artificial: $A = 1$.

Esse valor artificial é chamado de:

0-ésimo primo

A partir dele, o programa avança para o próximo primo B vezes.

Por isso:

```
B = 0 → A = 1  
B = 1 → A = 2  
B = 2 → A = 3  
B = 3 → A = 5
```

$B = 4 \rightarrow A = 7$
 $B = 5 \rightarrow A = 11$

Então, respondendo diretamente:

O primeiro primo real é 2.

O segundo primo real é 3.

O terceiro primo real é 5.

O material considera 1 como 0-ésimo primo apenas para ter um ponto inicial da enumeração e simplificar o algoritmo.

Exemplo 4.15 - Inteiros em sinal-magnitude

Um inteiro A é representado por dois registradores: A1 indica o sinal e A2 indica a magnitude. A1=0 representa número não negativo; A1=1 representa número negativo. Assim, +3 é (0,3), -3 é (1,3) e 0 é representado por (0,0).

Lógica de $A := A + 1$: se A é positivo ou zero, basta aumentar a magnitude. Se A é negativo, somar 1 reduz a magnitude; se a magnitude chegar a zero, o sinal deve voltar para positivo.

```
A := A + 1:
se A1 = 0 então
    A2 := A2 + 1
senão
    A2 := A2 - 1
    se A2 = 0 então
        A1 := A1 - 1
    senão
        √
```

Lógica de $A := A - 1$: se A é positivo e não zero, basta reduzir a magnitude. Se A é zero, A-1 vira -1, isto é, sinal negativo e magnitude 1. Se A já é negativo, subtrair 1 aumenta a magnitude.

```
A := A - 1:
se A1 = 0 então
    se A2 = 0 então
        A1 := A1 + 1
        A2 := A2 + 1
    senão
        A2 := A2 - 1
senão
    A2 := A2 + 1
```

O teste $A=0$ depende apenas da magnitude, pois na representação normalizada o zero é (0,0).

```
Teste A = 0:
se A2 = 0 então verdadeiro
senão falso
```

Exemplo 4.16 - Racionais

Um racional é representado como um par (a,b), com $b > 0$ e valor a/b . A representação não é única, pois $3/4$ e $6/8$ representam o mesmo racional. As operações são feitas por regras algébricas usuais de frações.

$(a,b) + (c,d) = (a.d + b.c, b.d)$
 $(a,b) - (c,d) = (a.d - b.c, b.d)$
 $(a,b) \cdot (c,d) = (a.c, b.d)$
 $(a,b) / (c,d) = (a.d, b.c), \text{ com } c \neq 0$
 $(a,b) = (c,d) \text{ se, e somente se, } a.d = b.c$

Exemplo 4.17 - Arranjo unidimensional

A ideia é armazenar todo o arranjo em um único registrador A. A posição n do arranjo fica no expoente do n-ésimo primo. Assim, incrementar A(n) equivale a multiplicar A por p_n ; decrementar A(n) equivale a dividir A por p_n se p_n divide A; testar se A(n)=0 equivale a verificar se p_n não divide A.

Para A(1)=5, A(2)=2, A(3)=0, A(4)=3, A(5)=1:

$A = 2^5 \cdot 3^2 \cdot 5^0 \cdot 7^3 \cdot 11^1 \cdot 13^0 \cdot 17^0 \dots$

Indexação direta:

```
adA(n) :  
    C := p_n  
    A := A . C  
  
subA(n) :  
    C := p_n  
    se teste_mod(A,C) então  
        A := div(A,C)  
    senão  
        √  
  
zeroA(n) :  
    C := p_n  
    se teste_mod(A,C) então  
        retorne falso  
    senão  
        retorne verdadeiro
```

Indexação indireta: a posição é o conteúdo do registrador B. Primeiro calculamos o B-ésimo primo.

```
adA(B) :  
    C := primo(B)  
    A := A . C  
  
subA(B) :  
    C := primo(B)  
    se teste_mod(A,C) então  
        A := div(A,C)  
    senão  
        √  
  
zeroA(B) :  
    C := primo(B)  
    se teste_mod(A,C) então  
        retorne falso  
    senão  
        retorne verdadeiro
```

===

Vamos organizar a explicação em três partes:

1. Como um arranjo é guardado em um único registrador
2. O que é indexação direta
3. O que é indexação indireta
4. Explicação passo a passo dos algoritmos adA, subA e zeroA

1. Ideia principal: como representar um vetor em um único registrador

Na Máquina Norma, cada registrador guarda apenas **um número natural**.

Então surge a pergunta:

Como guardar um arranjo, isto é, vários valores, dentro de um único registrador?

A estratégia usada no material é a **codificação por fatores primos**.

A ideia é representar cada posição do vetor usando um número primo diferente.

Por exemplo:

Posição do arranjo	Primo associado
A(1)	2
A(2)	3
A(3)	5
A(4)	7
A(5)	11
A(6)	13

Cada valor armazenado na posição vira o **expoente** do primo correspondente.

Exemplo do material

O material diz:

$$A(1) = 5$$

$$A(2) = 2$$

$$A(3) = 0$$

$$A(4) = 3$$

$$A(5) = 1$$

Então o registrador A armazena:

$$A = 2^5 \cdot 3^2 \cdot 5^0 \cdot 7^3 \cdot 11^1 \cdot 13^0 \cdot 17^0 \cdot \dots$$

Então o registrador **A** armazena:

$$A = 2^5 \cdot 3^2 \cdot 5^0 \cdot 7^3 \cdot 11^1 \cdot 13^0 \cdot 17^0 \cdot \dots$$

Vamos interpretar:

Posição	Valor armazenado	Primo	Trecho da codificação
A(1)	5	2	2^5
A(2)	2	3	3^2
A(3)	0	5	5^0
A(4)	3	7	7^3
A(5)	1	11	11^1

Portanto, o vetor:

$$A = [5, 2, 0, 3, 1]$$

é guardado como o número:

$$A = 2^5 \cdot 3^2 \cdot 5^0 \cdot 7^3 \cdot 11^1$$

Como $5^0 = 1$, ele não altera o produto.

Calculando parcialmente:

$$\begin{aligned} 2^5 &= 32 \\ 3^2 &= 9 \\ 5^0 &= 1 \\ 7^3 &= 343 \\ 11^1 &= 11 \end{aligned}$$

Logo:

$$\begin{aligned} A &= 32 \cdot 9 \cdot 1 \cdot 343 \cdot 11 \\ A &= 1.086.624 \end{aligned}$$

Então o número 1.086.624 representa o arranjo:

$A(1) = 5$
 $A(2) = 2$
 $A(3) = 0$
 $A(4) = 3$
 $A(5) = 1$

Como ler o valor de uma posição?

Para saber o valor de uma posição, olhamos quantas vezes o primo correspondente divide A .

Por exemplo, para saber $A(2)$, olhamos o primo da posição 2:

posição 2 $\rightarrow$ primo 3

Se A é divisível por 3^2 , mas não por 3^3 , então:

$A(2) = 2$

Porque o expoente do primo 3 é 2.

Outro exemplo:

posição 4 $\rightarrow$ primo 7

Se A é divisível por 7^3 , mas não por 7^4 , então:

$A(4) = 3$

3. O que significa adicionar 1 em uma posição?

Suponha o arranjo:

$A(1) = 5$
 $A(2) = 2$
 $A(3) = 0$
 $A(4) = 3$
 $A(5) = 1$

Codificado como:

$$A = 2^5 \cdot 3^2 \cdot 5^0 \cdot 7^3 \cdot 11^1$$

Agora queremos fazer:

$A(3) := A(3) + 1$

A posição 3 usa o primo 5.

Então, para aumentar $A(3)$ em 1, basta multiplicar o registrador A por 5.

Antes:

$$A = 2^5 \cdot 3^2 \cdot 5^0 \cdot 7^3 \cdot 11^1$$

Depois de multiplicar por 5:

$$A = 2^5 \cdot 3^2 \cdot 5^1 \cdot 7^3 \cdot 11^1$$

Agora:

$$A(3) = 1$$

Portanto:

Adicionar 1 em $A(n)$ equivale a multiplicar A pelo n -ésimo primo.

4. O que significa subtrair 1 de uma posição?

Agora suponha que temos:

$$A = 2^5 \cdot 3^2 \cdot 5^0 \cdot 7^3 \cdot 11^1$$

Queremos fazer:

$$A(2) := A(2) - 1$$

A posição 2 usa o primo 3.

Como $A(2) = 2$, o fator correspondente é:

$$3^2$$

Para diminuir o expoente de 3 de 2 para 1, basta dividir A por 3.

Antes:

$$A = 2^5 \cdot 3^2 \cdot 5^0 \cdot 7^3 \cdot 11^1$$

Depois de dividir por 3:

$$A = 2^5 \cdot 3^1 \cdot 5^0 \cdot 7^3 \cdot 11^1$$

Agora:

$$A(2) = 1$$

Portanto:

Subtrair 1 de $A(n)$ equivale a dividir A pelo n -ésimo primo.

Mas só podemos fazer isso se A for divisível por esse primo.

Se $A(n) = 0$, o primo correspondente não divide A .

5. O que significa testar se uma posição é zero?

Suponha:

$$A = 2^5 \cdot 3^2 \cdot 5^0 \cdot 7^3 \cdot 11^1$$

Queremos saber se:

$$A(3) = 0$$

A posição 3 usa o primo 5.

Como o expoente de 5 é zero:

$$5^0 = 1$$

isso significa que 5 não divide A .

Então:

$$A(3) = 0$$

se, e somente se:

A não é divisível pelo primo da posição 3.

Portanto:

Testar se $A(n) = 0$ equivale a testar se A não é divisível por p_n .

6. Indexação direta

Agora vamos ao primeiro caso do slide.

O que é indexação direta?

Na **indexação direta**, a posição do arranjo é conhecida previamente.

Por exemplo:

A(1)
A(2)
A(3)
A(4)

Quando escrevemos:

adA(3)

queremos dizer:

adicione 1 diretamente na posição 3 do arranjo A.

O índice 3 já aparece explicitamente no programa.

Por isso o algoritmo pode usar diretamente o primo correspondente à posição 3.

Programa Iterativo adA(n) — indexação direta

O slide mostra:

$C := p_n;$
 $A := A \cdot C$

Aqui:

p_n

significa o primo associado à posição n .

Por exemplo:

n **p_n**

1 2

2 3

3 5

4 7

5 11

Então:

adA(n)

faz:

$$A(n) := A(n) + 1$$

Exemplo de $\text{adA}(3)$

Considere:

$$\begin{aligned} A(1) &= 5 \\ A(2) &= 2 \\ A(3) &= 0 \\ A(4) &= 3 \\ A(5) &= 1 \end{aligned}$$

Logo:

$$A = 2^5 \cdot 3^2 \cdot 5^0 \cdot 7^3 \cdot 11^1$$

Queremos executar:

$$\text{adA}(3)$$

Como:

$$p_3 = 5$$

o algoritmo faz:

$$\begin{aligned} C &:= 5 \\ A &:= A \cdot C \end{aligned}$$

Antes:

$$A = 2^5 \cdot 3^2 \cdot 5^0 \cdot 7^3 \cdot 11^1$$

Depois:

$$A = 2^5 \cdot 3^2 \cdot 5^1 \cdot 7^3 \cdot 11^1$$

Portanto:

$A(3)$ passou de 0 para 1.

O arranjo passa de:

$$[5, 2, 0, 3, 1]$$

para:

$$[5, 2, 1, 3, 1]$$

7. Programa Iterativo $\text{subA}(n)$ — indexação direta

O slide mostra:

```
C := pn;  
  
se teste_mod(A, C)  
então A := div(A, C)  
senão ↓
```

Vamos traduzir.

Primeiro:

```
C := pn
```

Pega o primo da posição n .

Depois:

```
teste_mod(A, C)
```

testa se A é divisível por C .

Se A é divisível por C , significa que o expoente daquele primo é pelo menos 1.

Em outras palavras:

```
A(n) > 0
```

Então podemos subtrair 1 daquela posição fazendo:

```
A := div(A, C)
```

Ou seja:

```
A := A / C
```

Se A não for divisível por C , isso significa:

```
A(n) = 0
```

Nesse caso, não podemos subtrair 1, porque a Máquina Norma trabalha com números naturais.

Então o programa entra em indefinição, representada por:

↓

Esse símbolo significa que a operação não é possível, ou o programa não retorna normalmente.

Exemplo de `subA(2)`

Considere novamente:

$$A = 2^5 \cdot 3^2 \cdot 5^0 \cdot 7^3 \cdot 11^1$$

O arranjo é:

[5, 2, 0, 3, 1]

Queremos executar:

`subA(2)`

A posição 2 usa o primo:

$$p_2 = 3$$

Então:

`C := 3`

Agora testamos:

`teste_mod(A, 3)`

Como A possui o fator 3^2 , então A é divisível por 3.

Logo:

`teste_mod(A, 3) = verdadeiro`

Então executamos:

`A := div(A, 3)`

Antes:

$$A = 2^5 \cdot 3^2 \cdot 5^0 \cdot 7^3 \cdot 11^1$$

Depois:

$$A = 2^5 \cdot 3^1 \cdot 5^0 \cdot 7^3 \cdot 11^1$$

Portanto:

`A(2)` passou de 2 para 1.

O arranjo passou de:

[5, 2, 0, 3, 1]

para:

[5, 1, 0, 3, 1]

Exemplo de tentativa inválida: `subA(3)`

Agora considere:

$$A = 2^5 \cdot 3^2 \cdot 5^0 \cdot 7^3 \cdot 11^1$$

O arranjo é:

[5, 2, 0, 3, 1]

Queremos executar:

`subA(3)`

A posição 3 usa:

$$p_3 = 5$$

Então:

$$C := 5$$

Agora testamos:

`teste_mod(A, 5)`

Como o expoente de 5 em A é zero, temos:

A não é divisível por 5

Logo:

`teste_mod(A, 5) = falso`

Isso significa:

$$A(3) = 0$$

Então não podemos fazer:

$$A(3) := A(3) - 1$$

porque daria:

$$0 - 1$$

que não pertence aos naturais.

Por isso o programa cai em:

↓

8. Programa Iterativo $\text{zeroA}(n)$ — indexação direta

O slide mostra:

```
C := pn;  
se teste_mod(A, C)  
então falso  
senão verdadeiro
```

Esse programa testa se:

$$A(n) = 0$$

Vamos entender.

Se:

$$\text{teste_mod}(A, C) = \text{verdadeiro}$$

isso significa que C divide A .

Ou seja, o primo da posição n aparece na fatoração de A .

Logo:

$$A(n) > 0$$

Então a pergunta:

$$A(n) = 0?$$

deve retornar:

falso

Por outro lado, se:

$$\text{teste_mod}(A, C) = \text{falso}$$

isso significa que C não divide A .

Logo, o expoente do primo C é zero.

Então:

$$A(n) = 0$$

e o teste retorna:

verdadeiro

Exemplo de `zeroA(3)`

Considere:

$$A = 2^5 \cdot 3^2 \cdot 5^0 \cdot 7^3 \cdot 11^1$$

O arranjo é:

[5, 2, 0, 3, 1]

Queremos testar:

`zeroA(3)`

A posição 3 usa:

$$p_3 = 5$$

Então:

`C := 5`

Agora:

`teste_mod(A, 5)`

Como A não é divisível por 5, temos:

`teste_mod(A, 5) = falso`

Portanto:

`zeroA(3) = verdadeiro`

Isso está correto, pois:

$$A(3) = 0$$

Exemplo de `zeroA(4)`

Agora queremos testar:

`zeroA(4)`

A posição 4 usa:

$p_4 = 7$

Então:

$C := 7$

Agora:

`teste_mod(A, 7)`

Como A possui o fator 7^3 , então A é divisível por 7.

Logo:

`teste_mod(A, 7) = verdadeiro`

Portanto o algoritmo retorna:

`falso`

Isso está correto, pois:

$A(4) = 3$

Logo:

$A(4)$ não é zero.

9. Resumo da indexação direta

Na indexação direta, o índice é conhecido no próprio programa.

Por exemplo:

`adA(3)`
`subA(2)`
`zeroA(4)`

O programa sabe exatamente qual posição acessar.

Operação	Significado	Como faz
<code>adA(n)</code>	$A(n) := A(n) + 1$	multiplica A por p_n
<code>subA(n)</code>	$A(n) := A(n) - 1$	divide A por p_n , se possível
<code>zeroA(n)</code>	testa se $A(n) = 0$	verifica se p_n divide A

10. Indexação indireta

Agora vamos para a segunda parte do slide.

O que é indexação indireta?

Na indexação indireta, o índice não aparece diretamente no programa.

Em vez disso, o índice está guardado em um registrador.

Por exemplo:

$B = 4$

Então a operação:

`adA(B)`

significa:

adicionar 1 na posição $A(4)$

Se depois:

$B = 2$

a mesma operação:

`adA(B)`

significa:

adicionar 1 na posição $A(2)$

Portanto, na indexação indireta, a posição acessada depende do conteúdo de outro registrador.

11. Diferença entre direta e indireta

Indexação direta

O índice está escrito diretamente:

`adA(3)`

Significa sempre:

adicione 1 em $A(3)$

Não muda durante a execução.

Indexação indireta

O índice está dentro de um registrador:

$adA(B)$

Se:

$B = 3$

então acessa:

$A(3)$

Se:

$B = 5$

então acessa:

$A(5)$

O mesmo programa pode acessar posições diferentes dependendo do valor de B .

12. Por que aparece $C := \text{primo}(B)$?

Na indexação indireta, não sabemos antecipadamente qual posição será acessada.

Então precisamos descobrir, durante a execução, qual primo corresponde à posição guardada em B .

O comando:

$C := \text{primo}(B)$

faz exatamente isso.

Ele calcula o primo associado à posição B .

Por exemplo:

B $\text{primo}(B)$ Interpretação

B	primo(B)	Interpretação
1	2	primo da posição 1
2	3	primo da posição 2
3	5	primo da posição 3
4	7	primo da posição 4
5	11	primo da posição 5

Então, se:

$B = 4$

o comando:

$C := \text{primo}(B)$

faz:

$C := 7$

Porque a posição 4 é representada pelo primo 7.

13. Cuidado com o caso $B = 0$

No exemplo anterior do material, ele considera:

$\text{primo}(0) = 1$

Isso foi usado como base artificial da enumeração.

Mas, para indexar arranjos do tipo:

$A(1), A(2), A(3), \dots$

normalmente usamos:

$B \geq 1$

Assim:

$B = 1 \rightarrow \text{primo}(B) = 2$

$B = 2 \rightarrow \text{primo}(B) = 3$

$B = 3 \rightarrow \text{primo}(B) = 5$

O valor:

$B = 0$

não costuma representar uma posição válida do arranjo nesse exemplo, a menos que o material esteja usando arranjos indexados a partir de zero.

Para o exemplo do slide, como ele escreve:

$A(1), A(2), \dots$

vamos considerar:

índices válidos: 1, 2, 3, ...

14. Programa Iterativo $\text{adA}(B)$ — indexação indireta

O slide mostra:

```
C := primo(B)
A := A · C
```

Vamos interpretar.

Primeiro:

```
C := primo(B)
```

Calcula o primo correspondente à posição armazenada em B .

Depois:

```
A := A · C
```

Multiplica o registrador A por esse primo.

Isso aumenta em 1 o expoente daquele primo.

Logo:

```
A(B) := A(B) + 1
```

Exemplo de $\text{adA}(B)$ com $B = 4$

Considere o arranjo:

$A = [5, 2, 0, 3, 1]$

Codificado como:

$$A = 2^5 \cdot 3^2 \cdot 5^0 \cdot 7^3 \cdot 11^1$$

Agora considere:

$$B = 4$$

Queremos executar:

$$\text{adA}(B)$$

Como:

$$B = 4$$

a posição acessada é:

$$A(4)$$

O algoritmo faz:

$$C := \text{primo}(B)$$

Como:

$$\text{primo}(4) = 7$$

temos:

$$C = 7$$

Depois:

$$A := A \cdot C$$

Ou seja:

$$A := A \cdot 7$$

Antes:

$$A = 2^5 \cdot 3^2 \cdot 5^0 \cdot 7^3 \cdot 11^1$$

Depois:

$$A = 2^5 \cdot 3^2 \cdot 5^0 \cdot 7^4 \cdot 11^1$$

Portanto:

$A(4)$ passou de 3 para 4.

O arranjo passou de:

[5, 2, 0, 3, 1]

para:

[5, 2, 0, 4, 1]

Exemplo de $\text{adA}(B)$ com $B = 3$

Agora suponha:

$B = 3$

O comando:

$C := \text{primo}(B)$

faz:

$C := \text{primo}(3)$
 $C := 5$

Então:

$A := A \cdot 5$

Isso aumenta a posição 3.

Se antes:

$A = [5, 2, 0, 3, 1]$

depois:

$A = [5, 2, 1, 3, 1]$

15. Programa Iterativo $\text{subA}(B)$ — indexação indireta

O slide mostra:

$C := \text{primo}(B)$

se $\text{teste_mod}(A, C)$
então $A := \text{div}(A, C)$
senão ↓

A lógica é a mesma da indexação direta.

A diferença é que agora o primo não é conhecido antecipadamente.

Ele é calculado a partir de B .

Exemplo de $\text{subA}(B)$ com $B = 5$

Considere:

$$A = [5, 2, 0, 3, 1]$$

Codificado como:

$$A = 2^5 \cdot 3^2 \cdot 5^0 \cdot 7^3 \cdot 11^1$$

Agora:

$$B = 5$$

Queremos executar:

$$\text{subA}(B)$$

Primeiro:

$$C := \text{primo}(B)$$

Como:

$$\text{primo}(5) = 11$$

temos:

$$C = 11$$

Agora testamos:

$$\text{teste_mod}(A, 11)$$

Como A possui o fator 11^1 , então:

$$\text{teste_mod}(A, 11) = \text{verdadeiro}$$

Logo, podemos dividir:

$$A := \text{div}(A, 11)$$

Antes:

$$A = 2^5 \cdot 3^2 \cdot 5^0 \cdot 7^3 \cdot 11^1$$

Depois:

$$A = 2^5 \cdot 3^2 \cdot 5^0 \cdot 7^3 \cdot 11^0$$

Portanto:

$A(5)$ passou de 1 para 0.

O arranjo passou de:

[5, 2, 0, 3, 1]

para:

[5, 2, 0, 3, 0]

Exemplo inválido de $\text{subA}(B)$ com $B = 3$

Considere novamente:

$A = [5, 2, 0, 3, 1]$

Agora:

$B = 3$

A posição acessada será:

$A(3)$

Primeiro:

$C := \text{primo}(3)$

Logo:

$C = 5$

Agora testamos:

$\text{teste_mod}(A, 5)$

Como $A(3) = 0$, o fator 5 não aparece em A.

Então:

$\text{teste_mod}(A, 5) = \text{falso}$

Logo, não podemos dividir A por 5.

Isso corresponde a tentar fazer:

$A(3) := A(3) - 1$

ou seja:

$0 := 0 - 1$

Como isso não é permitido em números naturais, o programa vai para:

↓

16. Programa Iterativo `zeroA(B)` — indexação indireta

O slide mostra:

```
C := primo(B)
se teste_mod(A, C)
então falso
senão verdadeiro
```

A pergunta feita é:

$A(B) = 0?$

Ou seja:

A posição indicada pelo registrador `B` está zerada?

Primeiro o programa calcula:

```
C := primo(B)
```

Depois testa se `C` divide `A`.

Se divide, então o expoente daquele primo é maior que zero.

Logo:

$A(B) > 0$

Então:

```
zeroA(B) = falso
```

Se não divide, então o expoente é zero.

Logo:

$A(B) = 0$

Então:

`zeroA(B) = verdadeiro`

Exemplo de `zeroA(B)` com `B = 3`

Considere:

`A = [5, 2, 0, 3, 1]`

e:

`B = 3`

Primeiro:

`C := primo(B)`

Como:

`primo(3) = 5`

temos:

`C = 5`

Agora:

`teste_mod(A, 5)`

Como `A` não é divisível por 5, o teste retorna:

`falso`

Então o algoritmo retorna:

`verdadeiro`

Conclusão:

`zeroA(B) = verdadeiro`

porque:

`A(3) = 0`

Exemplo de `zeroA(B)` com `B = 4`

Agora:

`B = 4`

Primeiro:

```
C := primo(4)
```

Logo:

```
C = 7
```

Agora:

```
teste_mod(A, 7)
```

Como A possui o fator 7^3 , então A é divisível por 7.

Logo:

```
teste_mod(A, 7) = verdadeiro
```

Então o algoritmo retorna:

```
falso
```

Conclusão:

```
zeroA(B) = falso
```

porque:

```
A(4) = 3
```

e, portanto, não é zero.

17. Comparação direta entre indexação direta e indireta

Vamos usar o mesmo arranjo:

```
A = [5, 2, 0, 3, 1]
```

Codificado como:

$$A = 2^5 \cdot 3^2 \cdot 5^0 \cdot 7^3 \cdot 11^1$$

Acesso à posição 4 por indexação direta

Escrevemos explicitamente:

```
adA(4)
```

O programa sabe que precisa usar:

$p_4 = 7$

Então faz:

$A := A \cdot 7$

Resultado:

A(4) passa de 3 para 4

Acesso à posição 4 por indexação indireta

Agora escrevemos:

$adA(B)$

e o valor de B é:

$B = 4$

O programa primeiro calcula:

$C := \text{primo}(B)$
 $C := \text{primo}(4)$
 $C := 7$

Depois faz:

$A := A \cdot 7$

Resultado:

A(4) passa de 3 para 4

A operação final é a mesma.

A diferença é que, na indexação indireta, o índice veio de um registrador.

18. Tabela-resumo

Aspecto	Indexação direta	Indexação indireta
Como o índice aparece?	Escrito no programa	Guardado em um registrador
Exemplo	$adA(3)$	$adA(B)$

Aspecto	Indexação direta	Indexação indireta
Posição acessada	Sempre a posição 3	Depende do valor de B
Como obtém o primo?	Usa diretamente p_n	Calcula $\text{primo}(B)$
Flexibilidade	Menor	Maior
Exemplo prático	Alterar sempre $A(3)$	Alterar $A(B)$, onde B pode mudar

19. Exemplo completo com várias operações

Considere inicialmente:

$$A = [5, 2, 0, 3, 1]$$

Codificação:

$$A = 2^5 \cdot 3^2 \cdot 5^0 \cdot 7^3 \cdot 11^1$$

Operação 1: $\text{zeroA}(3)$ — direta

Queremos saber se:

$$A(3) = 0$$

Como:

$$p_3 = 5$$

testamos se 5 divide A.

Como 5 não divide A, então:

$$\text{zeroA}(3) = \text{verdadeiro}$$

Operação 2: $\text{adA}(3)$ — direta

Agora fazemos:

$$\text{adA}(3)$$

Isso multiplica A por 5.

Antes:

$$A = 2^5 \cdot 3^2 \cdot 5^0 \cdot 7^3 \cdot 11^1$$

Depois:

$$A = 2^5 \cdot 3^2 \cdot 5^1 \cdot 7^3 \cdot 11^1$$

Novo arranjo:

$$A = [5, 2, 1, 3, 1]$$

Operação 3: `zeroA(3)` novamente

Agora testamos:

```
zeroA(3)
```

Como agora A é divisível por 5, temos:

```
teste_mod(A, 5) = verdadeiro
```

Então:

```
zeroA(3) = falso
```

Porque agora:

$$A(3) = 1$$

Operação 4: `subA(2)` — direta

Agora fazemos:

```
subA(2)
```

A posição 2 usa:

$$p_2 = 3$$

Como A possui o fator 3^2 , podemos dividir por 3.

Antes:

$$A = 2^5 \cdot 3^2 \cdot 5^1 \cdot 7^3 \cdot 11^1$$

Depois:

$$A = 2^5 \cdot 3^1 \cdot 5^1 \cdot 7^3 \cdot 11^1$$

Novo arranjo:

$A = [5, 1, 1, 3, 1]$

Operação 5: $\text{adA}(B)$ com $B = 4$ — indireta

Agora:

$B = 4$

Executamos:

$\text{adA}(B)$

Primeiro:

$C := \text{primo}(4)$
 $C := 7$

Depois:

$A := A \cdot 7$

Antes:

$A = 2^5 \cdot 3^1 \cdot 5^1 \cdot 7^3 \cdot 11^1$

Depois:

$A = 2^5 \cdot 3^1 \cdot 5^1 \cdot 7^4 \cdot 11^1$

Novo arranjo:

$A = [5, 1, 1, 4, 1]$

20. Ideia final

A estrutura inteira funciona porque existe uma correspondência entre:

posição do arranjo $\leftrightarrow$ número primo

e entre:

valor armazenado na posição $\leftrightarrow$ expoente desse primo

Portanto:

$A(1), A(2), A(3), A(4), \dots$

vira:

$2^A(1) \cdot 3^A(2) \cdot 5^A(3) \cdot 7^A(4) \cdot \dots$

As operações ficam assim:

adA(n): multiplicar por p_n
subA(n): dividir por p_n , se possível
zeroA(n): testar se p_n divide A

Na **indexação direta**, o índice n já é conhecido.

Na **indexação indireta**, o índice está dentro de um registrador, por isso primeiro calculamos:

$C := \text{primo}(B)$

e depois usamos C para multiplicar, dividir ou testar divisibilidade.

Exemplo 4.18 - Pilha usando arranjo

Uma pilha segue a regra LIFO: o último valor inserido é o primeiro a sair. Para simulá-la em Norma, usamos um arranjo P para armazenar os elementos e um registrador T como índice do topo. Empilhar significa incrementar T e armazenar o valor no arranjo $P(T)$. Desempilhar significa recuperar $P(T)$, apagar essa posição e decrementar T .

```
empilha(V):  
    T := T + 1  
    P(T) := V  
  
desempilha(V):  
    se T = 0 então  
        erro ou V := 0  
    senão  
        V := P(T)  
        P(T) := 0  
        T := T - 1
```

Como $P(T)$ é uma posição de arranjo com indexação indireta, ela pode ser manipulada usando as macros do Exemplo 4.17.

Exemplo 4.19 - Endereçamento indireto em programa monolítico

Endereçamento indireto significa que o destino do desvio não é um rótulo fixo escrito no programa, mas o conteúdo de um registrador. A instrução r : faça F vá_para A deve executar F e depois desviar para a instrução cujo rótulo está armazenado em A.

Como programas monolíticos comuns têm destinos fixos, simulamos isso com uma macro End_A . A macro testa o valor de A e desvia para o rótulo correspondente. Ela funciona como uma tabela de despacho.

r : faça F vá_para A

Simulação:
 r : faça F vá_para End_A

```
Macro End_A:  
se A = 0 então vá_para 0  
senão decrementa A e testa se chegou a 0  
se chegou, vá_para 1  
senão decrementa novamente e testa  
se chegou, vá_para 2  
...  
se o valor original era k, vá_para k
```

Para um teste com endereçamento indireto, a forma geral é:

r: se T então vá_para A senão vá_para B

A simulação usa duas macros de despacho: se T for verdadeiro, chama End_A; se T for falso, chama End_B.

Parte B - Exercícios do PDF resolvidos

Exercício 4.1 - Codificação de programa monolítico

Convertendo cada instrução em quádrupla e depois em código:

Rótulo	Quádrupla	Código
1	(0, 1, 2, 2)	3675
2	(1, 1, 3, 1)	5250
3	(0, 2, 4, 4)	13505625
4	(1, 2, 1, 5)	1512630
5	(0, 3, 6, 6)	49633171875
6	(1, 2, 0, 5)	302526

Portanto, a codificação é:

$$P = 2^{3675} \cdot 3^{5250} \cdot 5^{13505625} \cdot 7^{1512630} \cdot 11^{49633171875} \cdot 13^{302526}$$

Resposta: alternativa c.

Exercício 4.2 - Codificação de programa monolítico

Rótulo	Quádrupla	Código
1	(1, 1, 2, 3)	51450
2	(0, 1, 3, 3)	128625
3	(0, 2, 4, 4)	13505625
4	(1, 1, 1, 0)	30

$$P = 2^{51450} \cdot 3^{128625} \cdot 5^{13505625} \cdot 7^{30}$$

Resposta: alternativa a.

Exercício 4.3 - Identificação dos expoentes

Rótulo	Quádrupla	Código
1	(1, 1, 4, 3)	1286250
2	(0, 1, 1, 1)	105
3	(1, 1, 2, 1)	1050

$$p = 2^{1286250} \cdot 3^{105} \cdot 5^{1050}$$

Logo, $x=1286250$, $y=105$ e $z=1050$. Resposta: alternativa b.

Exercício 4.4 - Quádrupla que não corresponde

As quádruplas do programa são:

Rótulo	Quádrupla
1	(1, 1, 2, 3)
2	(0, 1, 6, 6)
3	(1, 2, 5, 4)
4	(0, 2, 0, 0)
5	(0, 1, 0, 0)
6	(1, 2, 4, 1)

A alternativa (0,1,1,0) não aparece no programa. Resposta: alternativa a.

Exercício 4.5 - Codificação correta

Rótulo	Quádrupla	Código fatorado
1	(0, 1, 2, 2)	$2^0 \cdot 3^1 \cdot 5^2 \cdot 7^2$
2	(1, 1, 1, 3)	$2^1 \cdot 3^1 \cdot 5^1 \cdot 7^3$
3	(0, 2, 4, 4)	$2^0 \cdot 3^2 \cdot 5^4 \cdot 7^4$
4	(1, 2, 5, 1)	$2^1 \cdot 3^2 \cdot 5^5 \cdot 7^1$

$$P = 2^{(2^0 \cdot 3^1 \cdot 5^2 \cdot 7^2) \cdot 3^{(2^1 \cdot 3^1 \cdot 5^1 \cdot 7^3)} \cdot 5^{(2^0 \cdot 3^2 \cdot 5^4 \cdot 7^4)} \cdot 7^{(2^1 \cdot 3^2 \cdot 5^5 \cdot 7^1)}}$$

Resposta: alternativa e.

Exercício 4.6 - Número natural associado ao programa

Rótulo	Quádrupla	Código
--------	-----------	--------

1	(0, 1, 2, 2)	3675
2	(1, 1, 1, 3)	10290
3	(0, 2, 0, 0)	9

$P = 2^{3675} \cdot 3^{10290} \cdot 5^9$

Resposta: alternativa c.

Exercício 4.7 - Número que representa codificação de programa

Para representar um programa monolítico, cada expoente externo deve ser código válido de instrução. Uma instrução válida é teste, se começa com 2^1 , ou operação, se começa com 2^0 e possui os dois destinos iguais.

A alternativa d possui expoentes que decodificam como instruções válidas:

$8505 = 2^0 \cdot 3^5 \cdot 5^1 \cdot 7^1 \rightarrow$ operação (0,5,1,1)

$13230 = 2^1 \cdot 3^3 \cdot 5^1 \cdot 7^2 \rightarrow$ teste (1,3,1,2)

$2835 = 2^0 \cdot 3^4 \cdot 5^1 \cdot 7^1 \rightarrow$ operação (0,4,1,1)

Resposta: alternativa d.

Exercício 4.8 - Número que não representa codificação de instrução

Fatorando as alternativas:

$210 = 2^1 \cdot 3^1 \cdot 5^1 \cdot 7^1 \rightarrow$ teste válido

$450 = 2^1 \cdot 3^2 \cdot 5^2 \cdot 7^0 \rightarrow$ teste válido

$315 = 2^0 \cdot 3^2 \cdot 5^1 \cdot 7^1 \rightarrow$ operação válida

$750 = 2^1 \cdot 3^1 \cdot 5^3 \cdot 7^0 \rightarrow$ teste válido

$525 = 2^0 \cdot 3^1 \cdot 5^2 \cdot 7^1 \rightarrow$ seria operação, mas destinos 2 e 1 são diferentes

Como uma operação precisa ter a forma (0,k,r2,r2), 525 não representa uma instrução válida. Resposta: alternativa e.

Exercício 4.9 - Reescrita como programa iterativo

Decodificando $p = 2^{51450} \cdot 3^{105} \cdot 5^9$:

$51450 = 2^1 \cdot 3^1 \cdot 5^2 \cdot 7^3 \rightarrow$ (1,1,2,3)

$105 = 2^0 \cdot 3^1 \cdot 5^1 \cdot 7^1 \rightarrow$ (0,1,1,1)

$9 = 2^0 \cdot 3^2 \cdot 5^0 \cdot 7^0 \rightarrow$ (0,2,0,0)

O programa decodificado é:

1: se T então vá_para 2 senão vá_para 3

2: faça F vá_para 1

3: faça G vá_para 0

Isso corresponde a: enquanto T faça F; depois G. Resposta: alternativa d.

Exercício 4.10 - Codificação de programa iterativo

O programa iterativo é traduzido para instruções rotuladas assim:

1: se T1 então vá_para 2 senão vá_para 3

2: faça F1 vá_para 1

3: faça F2 vá_para 4

4: se T2 então vá_para 5 senão vá_para 0

5: se T1 então vá_para 0 senão vá_para 6

6: faça F3 vá_para 7

7: faça F1 vá_para 5

Rótulo	Quádrupla	Código fatorado
1	(1, 1, 2, 3)	$2^1 \cdot 3^1 \cdot 5^2 \cdot 7^3$
2	(0, 1, 1, 1)	$2^0 \cdot 3^1 \cdot 5^1 \cdot 7^1$
3	(0, 2, 4, 4)	$2^0 \cdot 3^2 \cdot 5^4 \cdot 7^4$
4	(1, 2, 5, 0)	$2^1 \cdot 3^2 \cdot 5^5 \cdot 7^0$
5	(1, 1, 0, 6)	$2^1 \cdot 3^1 \cdot 5^0 \cdot 7^6$
6	(0, 3, 7, 7)	$2^0 \cdot 3^3 \cdot 5^7 \cdot 7^7$
7	(0, 1, 5, 5)	$2^0 \cdot 3^1 \cdot 5^5 \cdot 7^5$

Essa sequência corresponde à alternativa b.

Exercício 4.11 - Codificação do fluxograma da Figura 4.5

O fluxograma executa F e depois testa T. Se T for verdadeiro, volta para o início; se T for falso, para. Logo:

```

1: faça F vá_para 2
2: se T então vá_para 1 senão vá_para 0

```

Códigos:

```

i1 = 2^0 . 3^1 . 5^2 . 7^2 = 3675
i2 = 2^1 . 3^1 . 5^1 . 7^0 = 30
P = 2^3675 . 3^30

```

Exercício 4.12 - Codificação do programa da Figura 4.6

Rótulo	Quádrupla	Código
1	(0, 1, 2, 2)	3675
2	(1, 1, 3, 1)	5250
3	(0, 2, 4, 4)	13505625
4	(1, 1, 1, 0)	30

```

P = 2^3675 . 3^5250 . 5^13505625 . 7^30

```

Exercício 4.13 - Simulador de Máquina Norma

A solução consiste em representar o programa como uma tabela de instruções rotuladas. Cada instrução possui tipo, registrador/operação/teste e destino. O simulador mantém um dicionário de registradores naturais e um contador de programa pc. A cada passo, busca a instrução do rótulo atual e executa.

```

# Simulador simples de Máquina Norma
class Norma:
    def __init__(self, program, entrada=0):
        self.program = program
        self.reg = {'X': entrada, 'Y': 0}
        self.pc = 1

    def get(self, r):
        return self.reg.get(r, 0)

    def set(self, r, v):
        self.reg[r] = max(0, v)

    def step(self):
        if self.pc == 0:
            return False
        inst = self.program[self.pc]
        tipo = inst[0]
        if tipo == 'ad':
            _, r, prox = inst
            self.set(r, self.get(r) + 1)
            self.pc = prox
        elif tipo == 'sub':
            _, r, prox = inst
            self.set(r, self.get(r) - 1)
            self.pc = prox
        elif tipo == 'zero':
            _, r, v, f = inst
            self.pc = v if self.get(r) == 0 else f
        return True

    def run(self, limite=100000):
        passos = 0
        while self.pc != 0 and passos < limite:
            self.step()
            passos += 1
        return self.get('Y')

```

Para calcular fatorial de 3, em uma versão didática com macros já definidas, usamos:

```

Y := 1
A := X
ate A = 0 faça
    Y := Y . A

```

```
A := A - 1
```

Entrada X=3 -> Y=6

A forma acima usa a macro de multiplicação, que pode ser expandida em operações primitivas conforme o Exemplo 4.10.

Exercício 4.14 - Macro para se A < 2

A condição $A < 2$ é verdadeira quando $A=0$ ou $A=1$. Para preservar A, copiamos A para C e testamos C.

Macro: r1: se A < 2 então vá_para r2 senão vá_para r3

```
C := A usando E
se C = 0 então vá_para r2
senão
  C := C - 1
  se C = 0 então vá_para r2
  senão vá_para r3
```

Exercício 4.15 - Programas em Norma para operações e testes

4.15(a) $A := B - C$

Em registradores naturais, a subtração é saturada: se $C > B$, o resultado fica 0. Para preservar B e C, copiamos ambos para registradores auxiliares.

```
A := B - C usando D,E,F:
A := B usando F
D := C usando F
ate D = 0 faça
  A := A - 1
  D := D - 1
```

4.15(b) $\text{div}(B,C)$: divisão inteira de B por C

A divisão inteira conta quantas vezes C cabe dentro de B. Usamos R como resto parcial e Q como quociente. Se $C=0$, a divisão não é definida; adotaremos $Q:=0$.

```
div(B,C) retorna Q usando R,D,E:
Q := 0
se C = 0 então
  Q := 0
senão
  R := B usando E
  enquanto C <= R faça
    R := R - C
    Q := Q + 1
retorne Q
```

4.15(c) $\text{fat}(A)$: fatorial

O fatorial de A é o produto $A \cdot (A-1) \cdot \dots \cdot 1$. Guardamos o valor original de A em N, fazemos $A:=1$ e multiplicamos A por N enquanto N não chega a zero.

```
fat(A) usando N,C,D:
N := A usando C
A := 1
ate N = 0 faça
  A := A . N usando C,D
  N := N - 1
```

4.15(d) $\text{pot}(X,Y)$: potência

A potência X^Y é obtida multiplicando o resultado por X exatamente Y vezes. Para $Y=0$, o resultado é 1.

```
pot(X,Y) retorna P usando E,C,D:
P := 1
```

```

E := Y usando C
ate E = 0 faca
    P := P . X usando C,D
    E := E - 1
retorne P

```

4.15(e) teste(A > B)

```

teste(A > B):
    retorne teste(B < A)

```

4.15(f) teste(A >= B)

```

teste(A >= B):
    se teste(A < B) então falso
    senão verdadeiro

```

4.15(g) teste(A <= B)

```

teste(A <= B):
    se teste(B < A) então falso
    senão verdadeiro

```

4.15(h) mdc(A,B)

Usamos o algoritmo de Euclides: $\text{mdc}(A,B) = \text{mdc}(B, A \bmod B)$ até $B=0$.

```

mdc(A,B) retorna M usando X,Y,R:
X := A
Y := B
ate Y = 0 faca
    R := mod(X,Y)
    X := Y
    Y := R
M := X

```

4.15(i) mod(A,B)

O resto é o que sobra de A após subtrair B repetidamente enquanto ainda for possível.

```

mod(A,B) retorna R usando D,E:
R := A
se B = 0 então
    R := 0
senão
    enquanto B <= R faça
        R := R - B
retorne R

```

4.15(j) primos(A,B): todos os primos entre A e B

Como a Máquina Norma retorna números naturais, representamos a lista de primos em um arranjo P codificado por primos. I é o índice da próxima posição livre.

```

primos(A,B) retorna arranjo P usando C,I:
P := 0
I := 1
C := A
ate B < C faca
    se teste_primo(C) então
        P(I) := C
        I := I + 1
    C := C + 1
retorne P

```

4.15(k) teste_nperf(A): número perfeito

Um número perfeito é igual à soma de seus divisores próprios positivos. Para $A=6$, divisores próprios são 1,2,3 e a soma é 6.

```

teste_nperf(A) usando D,S,R:
se A = 0 então falso
S := 0
D := 1
ate A <= D faça
    R := mod(A,D)
    se R = 0 então
        S := S + D
    D := D + 1
se S = A então verdadeiro
senão falso

```

Exercício 4.16 - Operações nos inteiros em sinal-magnitude

Representaremos cada inteiro K por dois registradores: K1 para o sinal e K2 para a magnitude. K1=0 indica não negativo; K1=1 indica negativo. As operações abaixo usam as macros naturais já definidas.

4.16(a) $A := B + C$

```

Se B1 = C1 então
    A1 := B1
    A2 := B2 + C2
senão
    se B2 > C2 então
        A1 := B1
        A2 := B2 - C2
    senão se C2 > B2 então
        A1 := C1
        A2 := C2 - B2
    senão
        A1 := 0
        A2 := 0

```

Lógica: sinais iguais somam magnitudes; sinais diferentes subtraem magnitudes e o resultado fica com o sinal da maior magnitude.

4.16(b) $A := B \cdot C$

```

A2 := B2 . C2
se A2 = 0 então
    A1 := 0
senão
    se B1 = C1 então
        A1 := 0
    senão
        A1 := 1

```

Lógica: o módulo do produto é o produto dos módulos. O sinal é negativo somente quando os sinais dos fatores são diferentes.

4.16(c) $A := B - C$

```

D1 := C1
D2 := C2
se D2 = 0 então
    D1 := 0
senão
    se D1 = 0 então D1 := 1
    senão D1 := 0
A := B + D

```

Lógica: $B-C$ é $B+(-C)$. Basta trocar o sinal de C, quando C não é zero, e usar a soma de inteiros.

4.16(d) $A := \text{div}(B,C)$

```

se C2 = 0 então
    erro ou A := 0
senão

```

```

A2 := div(B2,C2)
se A2 = 0 então
  A1 := 0
senão se B1 = C1 então
  A1 := 0
senão
  A1 := 1

```

Lógica: a magnitude é a divisão inteira das magnitudes; o sinal segue a regra dos sinais da multiplicação/divisão.

Exercício 4.17 - Operações nos racionais

Representaremos $B=(b_1,b_2)$ e $C=(c_1,c_2)$, com denominadores $b_2>0$ e $c_2>0$. Os numeradores podem ser inteiros, usando a representação do Exercício 4.16 se necessário.

4.17(a) $A := B + C$

```

A_num := b1 . c2 + b2 . c1
A_den := b2 . c2
A := (A_num, A_den)

```

4.17(b) $A := B . C$

```

A_num := b1 . c1
A_den := b2 . c2
A := (A_num, A_den)

```

4.17(c) $A := B - C$

```

A_num := b1 . c2 - b2 . c1
A_den := b2 . c2
A := (A_num, A_den)

```

4.17(d) $A := \text{div}(B,C)$

```

se c1 = 0 então
  erro: divisão por zero
senão
  A_num := b1 . c2
  A_den := b2 . c1
  se A_den < 0 então
    A_num := -A_num
    A_den := -A_den
  A := (A_num, A_den)

```

Opcionalmente, após cada operação pode-se simplificar a fração dividindo numerador e denominador pelo mdc de suas magnitudes.

Exercício 4.18 - NormaNeg equivalente à Norma

A Máquina NormaNeg permite registradores inteiros. Para simular isso em Norma, codificamos cada inteiro em um natural. Uma codificação possível é:

```

cod(z) = 2z, se z >= 0
cod(z) = 2|z| - 1, se z < 0

```

Assim: 0 -> 0, 1 -> 2, 2 -> 4, -1 -> 1, -2 -> 3, -3 -> 5. O teste $z=0$ é simplesmente teste se o código é 0.

Para simular incremento e decremento inteiro:

```

Incrementar z, isto é, z := z + 1:
se código é par então
  código := código + 2
senão
  se código = 1 então código := 0
  senão código := código - 2

```

```

Decrementar z, isto é, z := z - 1:
se código é par então
  se código = 0 então código := 1

```

```

senão código := código - 2
senão
  código := código + 2

```

Como somar ou subtrair 2 pode ser feito por duas operações ad ou sub da Norma, cada instrução de NormaNeg pode ser substituída por um trecho de programa Norma. Portanto, todo programa NormaNeg pode ser simulado por Norma usando a mesma quantidade de registradores, apenas mudando a codificação interna dos valores.

Exercício 4.19 - Programa que realiza $A := B$

Com $A=0$ e $B=5$, queremos que A receba 5. A alternativa correta é a que transfere uma unidade de B para A até B chegar a zero:

```

1: se zeroB então vá_para 5 senão vá_para 2
2: faça adA vá_para 3
3: faça subB vá_para 1

```

Essa é a alternativa d. A cada ciclo, A aumenta uma unidade e B diminui uma unidade. Após cinco ciclos, $A=5$ e $B=0$. Observação: essa versão não preserva B; se fosse necessário preservar B, seria preciso usar um registrador auxiliar como no Exemplo 4.9.

Exercício 4.20 - Característica correta da Máquina Norma

A Máquina Norma possui registradores naturais que podem assumir valores naturais arbitrariamente grandes, sem limite teórico de tamanho. Ela não usa fita como a Máquina de Turing, não permite valores quaisquer diretamente e consegue simular recursão.

Resposta: alternativa a.

Exercício 4.21 - Afirmações sobre poder computacional

I é verdadeira: a Máquina Norma é simples, mas possui poder computacional equivalente aos modelos universais de computação. II é verdadeira: sub-rotinas e recursão podem ser simuladas com endereçamento indireto. III é falsa: operações complexas podem ser construídas por macros, ainda que com programas longos.

Resposta: alternativa d, apenas I e II estão corretas.

Exercício 4.22 - Afirmações sobre memória e recursão

I é falsa, pois a memória da Máquina Norma é um conjunto infinito, embora contável, de registradores naturais. II é verdadeira, pois a Máquina Norma é universal. III é verdadeira, pois sub-rotinas e recursão podem ser simuladas por programas monolíticos usando endereçamento indireto.

Resposta: alternativa d, apenas II e III estão corretas.

Resumo final das respostas objetivas

Exercício	Resposta
4.1	c
4.2	a
4.3	b
4.4	a
4.5	e
4.6	c
4.7	d
4.8	e
4.9	d
4.10	b
4.11	$2^{3675} \cdot 3^{30}$
4.12	$2^{3675} \cdot 3^{5250} \cdot 5^{13505625} \cdot 7^{30}$
4.19	d
4.20	a
4.21	d
4.22	d